Studio Nx

S1-10 프로젝트 전체 저장·불러오기 API — 함수 설명 합본 (백엔드 264~268)

2026-10-19

목차

1. ai-notes/backend(fastAPI_Python)/S1-10 - 프로젝트 전체 저장·불러오기/264_schemas_full_project.md
2. ai-notes/backend(fastAPI_Python)/S1-10 - 프로젝트 전체 저장·불러오기/265_full_project_service_info.md
3. ai-notes/backend(fastAPI_Python)/S1-10 - 프로젝트 전체 저장·불러오기/266_full_project_service_get_full_project.md
4. ai-notes/backend(fastAPI_Python)/S1-10 - 프로젝트 전체 저장·불러오기/267_project_service_update_settings.md
5. ai-notes/backend(fastAPI_Python)/S1-10 - 프로젝트 전체 저장·불러오기/268_api_projects_full.md

Backend(FastAPI/Python) 264 — 프로젝트 전체 응답 스키마 (`full_project.py`)

항목	내용
파일	`studionx/backend/app/schemas/full_project.py`
작업	S1-10 프로젝트 전체 저장·불러오기: 전체 조회와 곡 설정 스키마 (PRD 02 FR-253, FR-254)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	263 `wireCells` 외 화면 배선 · 265 `_group_by` · `project_info`

1. 하는 일

시퀀서가 화면을 열 때 받는 한 덩어리 응답의 모양을 정합니다. 네 클래스를 한 파일에 둡니다.

```
GET /projects/{id}/full
└ FullProjectResponse
   ├ project  FullProjectInfo      곡 설정까지 담은 프로젝트 (bpm · 박자 · Key · master_settings · revis
ion)
   ├ tracks[] FullTrackResponse    트랙 한 줄 + instruments · effects · links
   ├ patterns[] PatternResponse    패턴 + notes (S1-08 스키마를 그대로 씁니다)
   └ clips · markers · tempo_changes · meter_changes   지금은 늘 빈 목록
```

곡 설정을 저장하는 요청의 모양(FullProjectSettingsUpdateRequest)도 여기 있습니다.

2. 언제 쓰나

쓰는 곳	
시퀀서 `sequencer.js` `loadProject`	`GET /projects/{id}/full` 의 응답을 이 모양으로 검사해 받습니다
시퀀서 수동 저장·자동 저장	바뀐 곡 설정만 `PATCH /projects/{id}/settings` 로 보냅니다
백엔드 라우터·서비스	`full_project_service` 가 이 클래스들을 채워 돌려줍니다

3. 코드

```
class FullProjectInfo(BaseModel):
    project_id: int
    title: str
    description: str | None
    bpm: Decimal
    time_sig_num: int
    time_sig_den: int
    music_key: str | None
    scale_mode: str | None
    sample_rate: int
    master_settings: dict[str, Any] | None
    revision: int
```

```
    my_role: str
    member_count: int
    opened: bool
    created_at: datetime
    updated_at: datetime


class FullTrackResponse(TrackResponse):
    instruments: list[TrackInstrumentResponse]
    effects: list[TrackPluginResponse]
    links: list[TrackPluginLinkResponse]


class FullProjectResponse(BaseModel):
    project: FullProjectInfo
    tracks: list[FullTrackResponse]
    patterns: list[PatternResponse]
    clips: list[dict[str, Any]] = Field(default_factory=list)
    markers: list[dict[str, Any]] = Field(default_factory=list)
    tempo_changes: list[dict[str, Any]] = Field(default_factory=list)
    meter_changes: list[dict[str, Any]] = Field(default_factory=list)


class FullProjectSettingsUpdateRequest(BaseModel):
    bpm: Decimal | None = Field(default=None, ge=BPM_MIN, le=BPM_MAX, decimal_places=2)
    time_sig_num: int | None = Field(default=None, ge=1, le=TIME_SIG_MAX)
    time_sig_den: int | None = Field(default=None, ge=1, le=TIME_SIG_MAX)
    music_key: str | None = Field(default=None, max_length=10)
    scale_mode: str | None = Field(default=None, max_length=20)
    sample_rate: int | None = Field(default=None, ge=SAMPLE_RATE_MIN, le=SAMPLE_RATE_MAX)
    master_settings: dict[str, Any] | None = None
```

문법 풀이

식	뜻
`class FullTrackResponse(TrackResponse)`	트랙 칸(231)을 그대로 물려받고 장치 세 목록만 더합니다. 트랙 칸 이름이 목록 API 와 어긋나지 않습니다
`Field(default_factory=list)`	기본값을 빈 목록으로 둡니다. `[]` 를 그대로 쓰면 같은 목록을 모든 응답이 나눠 쓰게 되어 위험합니다
`Decimal | None = Field(default=None, ge=..., le=...)`	보내지 않으면 `None` 이고, 보내면 범위를 검사합니다
`BPM_MIN` · `BPM_MAX`	20 과 300 입니다. 파일 위쪽에 상수로 두어 시험과 서비스가 같은 값을 봅니다
`master_settings: dict[str, Any] | None`	자유로운 모양의 마스터 설정 묶음입니다 (DB 칸이 JSON)

4. 왜 이렇게

선택	이유
응답을 한 클래스에 중첩	화면이 여섯 번 요청하던 것을 한 번으로 줄입니다 (FR-254)
트랙 스키마를 물려받음	트랙 칸 이름을 새로 적으면 목록 API 와 어긋날 수 있습니다

선택	이유
아직 없는 표는 빈 목록으로 내보냄	화면이 `clips` · `markers` 칸을 늘 있다고 보고 읽을 수 있습니다. 나중에 S2-04(오토메이션)와 클립 저장 작업이 이 목록만 채웁니다
`revision` 을 함께 내보냄	나중에 동시 편집(S3-02)이 어긋남을 알아채는 값입니다. 지금은 화면이 읽기만 합니다
곡 설정 수정은 보낸 칸만	화면이 Ctrl+S·자동 저장 때 바뀐 칸만 보내므로 나머지는 건드리지 않습니다

5. 시험

`tests/test_full_project_routes.py` 에서 응답의 칸 이름 집합(PROJECT_KEYS · TRACK_KEYS · PATTERN_KEYS · NOTE_KEYS)과 빈 목록 네 칸을 확인했습니다. `tests/test_project_settings_routes.py` 에서 범위를 벗어난 BPM(5 · 400) · 박자(0) · 샘플레이트(100)가 422 로 막히는지 확인했습니다. 모두 통과합니다.

Backend(FastAPI/Python) 265 — 묶기와 프로젝트 칸 (`_group_by` · `project_info`)

항목	내용
파일	`studionx/backend/app/services/full_project_service.py`
작업	S1-10 프로젝트 전체 저장·불러오기: 묶기 도우미와 프로젝트 칸 (PRD 02 FR-254)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	264 프로젝트 전체 응답 스키마 · 266 `_full_track` · `get_full_project`

1. 하는 일

전체 조회(266)를 도와주는 두 함수입니다.

```
_group_by(rows, key)      트랙별 장치 · 패턴별 노트로 묶습니다
project_info(db, ...)     프로젝트 + 곡 설정 + 내 역할 → FullProjectInfo
```

`project_info` 는 시퀀서가 화면 위쪽(BPM · 박자 · Key)을 그리는 데 쓰는 값을 만듭니다.

2. 언제 쓰나

쓰는 곳	
`get_full_project` (266)	트랙 · 악기 · 이펙트 · 연결선 · 노트를 묶을 때 `_group_by` 를 씁니다
`get_full_project` · 곡 설정 라우터 (268)	두 곳 모두 `project_info` 로 프로젝트 칸을 만듭니다
`PATCH /projects/{id}/settings` 응답	저장 뒤 바뀐 프로젝트 칸을 그대로 돌려줍니다

3. 코드

```
def _group_by(rows: list, key) -> dict[Any, list]:
    """행 목록을 key(row) 값으로 묶는다 — 트랙별 장치, 패턴별 노트. 차례는 조회 때 정한 차례를 지킨다."""
    grouped: dict[Any, list] = {}
    for row in rows:
        grouped.setdefault(key(row), []).append(row)
    return grouped


def project_info(db: Session, project: Project, member: ProjectMember) -> FullProjectInfo:
    """Project → FullProjectInfo. 멤버 수와 opened 는 project_service.to_item 이 센 값을 그대로 쓴다."""
    item = project_service.to_item(db, project, member)
    return FullProjectInfo(
        **item.model_dump(),
        scale_mode=project.scale_mode,
        sample_rate=project.sample_rate,
        master_settings=project.master_settings,
```

```
        revision=project.revision,
    )
```

문법 풀이

식	뜻
`key` 인자	함수 자체를 받습니다. `lambda row: row.track_id` 처럼 무엇으로 묶을지 넘깁니다
`dict[Any, list]`	열쇠가 트랙 번호(정수)이거나 패턴 번호(정수)라 둘 다 담도록 열쇠를 `Any` 로 둡니다
`grouped.setdefault(k, []).append(row)`	그 열쇠가 처음이면 빈 목록을 만들어 넣고, 목록 끝에 행을 더합니다
`**item.model_dump()`	목록 한 줄(065)의 칸을 전부 펼쳐 넣고, 그 뒤에 곡 설정 네 칸을 더합니다
`member.role`	`my_role` 에 들어가는 `'OWNER'` · `'EDITOR'` · `'VIEWER'` 값입니다

4. 왜 이렇게

선택	이유
트랙 수만큼 묻지 않고 묶어서 처리	화면을 열 때마다 곡 크기에 비례해 느려지면 안 됩니다. 조회 여섯 번으로 끝냅니다
`_group_by` 를 두 곳에서 씀	장치와 노트의 묶는 방법이 같아 함수 하나로 충분합니다
멤버 수 · opened 를 `to_item` 에 맡김	마이페이지 목록과 시퀀서가 다른 값을 보여 주면 버그로 보입니다. 세는 규칙을 한 곳에 둡니다
곡 설정만 따로 더함	`ProjectItem` 은 목록 화면용이라 `scale_mode` · `sample_rate` · `master_settings` · `revision` 이 없습니다

5. 시험

`tests/test_full_project_routes.py::test_full_includes_song_settings` 에서 BPM · Key · 스케일 · 마스터 설정 · revision · sample_rate 가 응답에 그대로 실리는지 확인했습니다.
`test_full_returns_project_tracks_and_patterns` 에서 `my_role` 이 `'OWNER'`, `member_count` 가 1 인지 확인했습니다. 통과합니다.

Backend(FastAPI/Python) 266 — 트랙 한 줄과 전체 조회 (`_full_track` · `get_full_project`)

항목	내용
파일	`studionx/backend/app/services/full_project_service.py`
작업	S1-10 프로젝트 전체 저장·불러오기: 전체 조회 서비스 (PRD 02 FR-254)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	265 `_group_by` · `project_info` · 267 `project_service.update_settings`

1. 하는 일

시퀀서가 화면을 열 때 부르는 한 번의 조회를 실제로 처리합니다.

```
get_full_project(db, user, project_id)
1. project_service.get_membership        멤버인지 확인 (아니면 404)
2. tracks       트랙 목록 (order_index 순)          조회 1번
3. instruments 악기 칸 (slot_index 순)              조회 1번 — 트랙 번호들로 한꺼번에
4. effects      이펙트 체인 (slot_index 순)          조회 1번
5. links        갈래 연결선                         조회 1번
6. patterns     패턴 목록                           조회 1번
7. notes        노트 (start_tick · pitch 순)         조회 1번
8. _group_by 로 트랙별 · 패턴별로 묶어 응답 조립
```

`_full_track` 은 그중 트랙 한 줄을 만드는 부분입니다.

2. 언제 쓰나

쓰는 곳	
`GET /projects/{id}/full` (268)	라우터가 이 함수를 그대로 부릅니다
시퀀서 화면 열기	트랙 · 악기 · 이펙트 · 패턴 · 노트를 한 번에 받아 그립니다 (FR-254)

3. 코드

```
def _full_track(track, instruments, effects, links) -> FullTrackResponse:
    """트랙 한 줄 + 그 트랙의 악기·이펙트·연결선 (FR-245 · 263 · 267 · 269)."""
    base = TrackResponse.model_validate(track).model_dump()
    return FullTrackResponse(
        **base,
        instruments=[TrackInstrumentResponse.model_validate(r) for r in instruments.get(track.track_id,
[])],
        effects=[TrackPluginResponse.model_validate(r) for r in effects.get(track.track_id, [])],
        links=[TrackPluginLinkResponse.model_validate(r) for r in links.get(track.track_id, [])],
    )
```

```
def get_full_project(db, user, project_id) -> FullProjectResponse:
    """프로젝트 전체 (FR-254). 멤버면 누구나 (VIEWER 포함)."""
    project, member = project_service.get_membership(db, project_id, user)

    tracks = list(db.scalars(
        select(Track).where(Track.project_id == project_id).order_by(Track.order_index, Track.track_id)
    ).all())
    track_ids = [t.track_id for t in tracks]
    if track_ids:
        instruments = _group_by(..., lambda row: row.track_id)
        effects = _group_by(..., lambda row: row.track_id)
        links = _group_by(..., lambda row: row.track_id)
    else:
        instruments, effects, links = {}, {}, {}

    patterns = list(db.scalars(
        select(Pattern).where(Pattern.project_id == project_id).order_by(Pattern.pattern_id)
    ).all())
    pattern_ids = [p.pattern_id for p in patterns]
    notes = _group_by(..., lambda row: row.pattern_id) if pattern_ids else {}

    return FullProjectResponse(
        project=project_info(db, project, member),
        tracks=[_full_track(t, instruments, effects, links) for t in tracks],
        patterns=[pattern_service.to_response(p, notes.get(p.pattern_id, [])) for p in patterns],
    )
```

문법 풀이

식	뜻
`db.scalars(select(...)).all()`	여러 행을 읽어 옵니다. `list(...)` 로 감싸 목록으로 만듭니다
`.in_(track_ids)`	트랙 번호 여러 개를 한 번의 조회로 가져옵니다 (`WHERE track_id IN (…)`)
`if track_ids:`	빈 목록으로 `IN ()` 을 보내면 오류가 나므로 트랙이 있을 때만 묻습니다
`instruments.get(track_id, [])`	그 트랙에 장치가 없으면 빈 목록입니다
`lambda row: row.track_id`	무엇으로 묶을지 알려 주는 작은 함수입니다
`pattern_service.to_response(pattern, notes)`	패턴 + 노트 → 응답 한 건 (215). 노트를 넘겨 관계 읽기를 건너뜁니다

4. 왜 이렇게

선택	이유
트랙 · 패턴을 먼저 읽고 장치를 한 꺼번에	트랙마다 서너 번 묻던 것을 조회 여섯 번으로 줄입니다
트랙이 없을 때 조회를 건너뜀	빈 `IN ()` 은 MySQL 이 싫어합니다. MASTER 는 늘 있지만 규칙으로 막아 둡니다
노트를 패턴과 함께 담음	피아노롤과 클립 미리보기가 곧바로 그릴 수 있습니다. 목록 API(217)는 가볍게 두려고 노트를 빼 둡니다
장치 차례를 slot_index 순으로	화면의 이펙트 체인 순서가 곧 `slot_index` 입니다 (FR-263)

선택	이유
없는 표는 담지 않음	clips · markers · automation 은 모델이 없어 조회할 수 없습니다. 응답 스키마(264)가 빈 목록으로 채웁니다

5. 시험

`tests/test_full_project_routes.py` 9건이 모두 통과합니다. `test_full_attaches_devices_to_their_own_track` 에서 Bass Synth 의 악기 · 이펙트가 Keys 의 것과 섞이지 않는지, 연결선이 Keys 에만 붙는지 확인했습니다. `test_full_without_devices_returns_empty_lists` 는 MASTER 한 줄만 있는 새 프로젝트에서도 빈 목록이 나오는지 봅니다. `test_full_viewer_can_read` 는 VIEWER 도 읽을 수 있는지, `test_full_of_someone_elses_project_is_404` · `test_full_unknown_project_is_404` 는 남의 것과 없는 번호를 404 로 막는지 확인합니다.

Backend(FastAPI/Python) 267 — 곡 설정 저장 (`project_service.update_settings`)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	S1-10 프로젝트 전체 저장·불러오기: 곡 전체 설정 저장 (PRD 02 FR-253)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	266 `get_full_project` · 268 프로젝트 라우터(전체 조회 · 곡 설정)

1. 하는 일

BPM · 박자 · Key · 스케일 · 샘플레이트 · 마스터 설정을 저장합니다. 화면의 Ctrl+S 수동 저장과 자동 저장이 바뀐 칸만 보내는 곳입니다.

```
PATCH /projects/{id}/settings { bpm: 140.5, music_key: 'Am' }
1. require_role  OWNER · EDITOR 인지 확인 (VIEWER 는 403)
2. 보낸 칸만 setattr (exclude_unset)
3. commit → refresh → Project 반환
```

2. 언제 쓰나

쓰는 곳	
`PATCH /projects/{id}/settings` (268)	라우터가 이 함수를 부릅니다
시퀀서 Ctrl+S · 자동 저장	화면이 고친 BPM · 박자 · Key 를 보냅니다 (FR-253)
마스터 설정 저장	마스터 트랙의 리미터 · 페이드 설정 묶음을 담습니다 (FR-252 이후 작업)

3. 코드

```
def update_settings(db, project, member, data) -> Project:
    """곡 전체 설정(BPM · 박자 · Key · 스케일 · 샘플레이트 · 마스터 설정) 수정 (FR-253) — OWNER · EDITOR 만."""
    require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    for name, value in data.model_dump(exclude_unset=True).items():
        setattr(project, name, value)
    db.commit()
    db.refresh(project)
    return project
```

문법 풀이

식	뜻
`require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	역할이 둘 중 하나가 아니면 403 을 냅니다 (056)

식	뜻
`model_dump(exclude_unset=True)`	요청에 실제로 담겨 온 칸만 사전으로 만듭니다. 보내지 않은 칸은 빠집니다
`setattr(project, name, value)`	칸 이름과 값으로 그대로 넣습니다
`db.commit()`	여기서 실제 DB 에 저장됩니다 (`get_db` 는 commit 을 하지 않습니다)
`db.refresh(project)`	DB 가 채운 `updated_at` 을 다시 읽습니다

4. 왜 이렇게

선택	이유
수정(`update_project`)과 함수를 나눔	제목 · 설명 수정(FR-124)은 화면이 다르고 응답도 다릅니다. 한 함수에 섞으면 권한 · 응답이 꼬입니다
`exclude_unset` 으로 바뀐 칸만	화면이 곡 설정 전체를 매번 보내면 다른 창에서 바꾼 값을 덮어씁니다 (S3-02 동시 편집 대비)
저장 뒤 프로젝트 칸을 돌려줌	화면이 저장 결과(새 `updated_at` · 값)를 곧바로 표시합니다
새 표를 만들지 않음	곡 설정은 이미 `projects` 칸으로 있습니다 (01_phase1_account.sql 83~88행)
`revision` 은 건드리지 않음	그 값은 협업 연산(FR-611)이 올립니다. 저장이 함부로 올리면 충돌 판정이 어긋납니다

5. 시험

`tests/test_project_settings_routes.py` 10건이 모두 통과합니다. 보낸 칸만 바뀌고 나머지는 그대로인지 (`test_settings_updates_only_sent_fields`), 마스터 설정과 박자를 담을 수 있는지, EDITOR 는 200 · VIEWER 는 403 · 남의 프로젝트는 404 인지, 범위를 벗어난 값이 422 인지 확인합니다.

Backend(FastAPI/Python) 268 — 프로젝트 라우터 (`GET /projects/{id}/full` · `PATCH /projects/{id}/settings`)

항목	내용
파일	`studionx/backend/app/api/v1/projects.py`
작업	S1-10 프로젝트 전체 저장·불러오기: 전체 조회와 곡 설정 라우터 (PRD 02 FR-253, FR-254)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	267 `update_settings` · 269 이후 화면 JS (`loadProject` · 명령 레지스트리)

1. 하는 일

P1-13 라우터(049 `p1-13_project_api`)에 두 길을 더합니다. 라우터는 얇게 두고 규칙은 서비스에 맡깁니다.

메서드 · 경로	하는 일	권한
`GET /projects/{id}/full`	프로젝트 + 트랙(+악기 · 이펙트 · 연결선) + 패턴(+노트)	멤버면 누구나
`PATCH /projects/{id}/settings`	곡 전체 설정(BPM · 박자 · Key · 스케일 · 샘플레이트 · 마스터)	OWNER · EDITOR

2. 언제 쓰나

쓰는 곳	
시퀀서를 열 때	`GET .../full` 한 번으로 화면 전체를 그립니다 (FR-254)
Ctrl+S · 자동 저장	바뀐 곡 설정만 `PATCH .../settings` 로 보냅니다 (FR-253)
곡 설정 응답	저장한 결과를 프로젝트 칸 모양으로 돌려주어 화면이 그대로 표시합니다

3. 코드

```
@router.get("/{project_id}/full", response_model=FullProjectResponse, responses=NOT_FOUND)
def read_full_project(project_id: int, user: CurrentUser, db: Db) -> FullProjectResponse:
    """프로젝트 전체 — 트랙·악기·이펙트·연결선·패턴·노트를 한 번에 (FR-254)."""
    return full_project_service.get_full_project(db, user, project_id)


@router.patch(
    "/{project_id}/settings",
    response_model=FullProjectInfo,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def update_project_settings(
    project_id: int,
    data: FullProjectSettingsUpdateRequest,
    user: CurrentUser,
    db: Db,
) -> FullProjectInfo:
    """곡 전체 설정 저장 (FR-253) — OWNER · EDITOR."""
```

```
    project, member = project_service.get_membership(db, project_id, user)
    project = project_service.update_settings(db, project, member, data)
    return full_project_service.project_info(db, project, member)
```

문법 풀이

식	뜻
`CurrentUser` · `Db`	`app/api/deps.py` 별칭입니다. 로그인한 사용자와 DB 세션을 자동으로 받습니다
`"/{project_id}/full"`	경로 뒤에 `full` 을 붙입니다. `GET /projects/{id}` (069) 와 겹치지 않습니다
`response_model=FullProjectInfo`	곡 설정 저장 뒤에도 프로젝트 칸 전체를 돌려줍니다
`responses={**NOT_FOUND, **FORBIDDEN}`	문서에 404 · 403 모양을 적어 둡니다
`**` 두 사전 합치기	`NOT_FOUND`(404) 와 `FORBIDDEN`(403) 을 한 사전으로 만듭니다

4. 왜 이렇게

선택	이유
라우터는 얇게	권한 · 404 규칙은 서비스 한 곳에 모읍니다. 라우터는 부르기만 합니다
전체 조회를 `/projects` 아래에 둠	프로젝트 하나의 상태라 프로젝트 자원 아래가 맞습니다. 트랙 · 패턴 라우터는 그대로 둡니다
곡 설정을 `settings` 경로로	제목 · 설명(`PATCH /projects/{id}`)과 섞지 않습니다. 시퀀서가 곡 설정만 보내기 쉽습니다
401 은 자동	`CurrentUser` 가 로그인을 요구합니다
응답이 프로젝트 칸	저장 직후 화면이 값을 다시 묻지 않아도 됩니다

5. 시험

`tests/test_full_project_routes.py` 의 `test_full_route_registered` 와 `tests/test_project_settings_routes.py` 의 `test_settings_route_registered` 가 `app.openapi()["paths"]` 에 두 경로가 실렸는지 확인합니다. 같은 파일들의 나머지 시험은 실제 앱 · MySQL 로 요청해 응답과 권한(200 · 401 · 403 · 404 · 422)을 확인하며 모두 통과합니다.